Protocols for Secure Communication

BCS2420

Dr. Ashish Sai
ashish.sai@maastrichtuniversity.nl

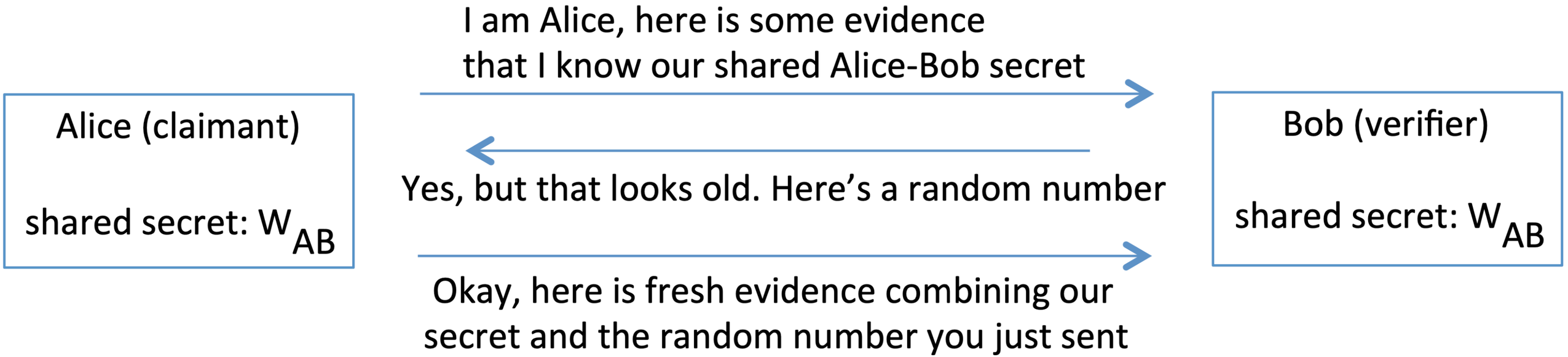
Entity Authentication and Key Establishment

Basic Concepts

- **Protocol**: Exchange of messages between parties (devices).
- **Entity Authentication**: Verifying the identity of a communicating party.
- **Cryptographic Protocol**: Protocol involving cryptographic techniques.
- **Authentication Protocol**: Provides entity authentication or authenticated key establishment.

Example: Browser-Server Authentication ✍️

- **Unilateral Authentication**: One party authenticates to another.
- **Mutual Authentication**: Both parties authenticate to each other.


I am Alice, here is some evidence
that I know our shared Alice-Bob secret
Alice (claimant)
shared secret: W_{AB}
Yes, but that looks old. Here's a random number
Bob (verifier)
shared secret: W_{AB}
Okay, here is fresh evidence combining our
secret and the random number you just sent

Session Keys

- **Key Establishment**: Arranging a shared secret (symmetric key) for secure communications.
  - **Session Keys**: Keys used for short-term purposes.

Key Transport vs. Key Agreement

- **Key Transport**: One party unilaterally chooses and transfers the symmetric key.
- **Key Agreement**: Shared key is a function of values contributed by both parties.

Authentication and Key Establishment Protocols

Types of Protocols

1. **Authentication-Only**: Provides identity assurance without establishing a session key.
2. **Unauthenticated Key Establishment**: Establishes a session key without identity assurance.

Integrating Authentication with Key Establishment 🔑

- **Authenticated Key Establishment**: Combining both functions in one protocol.

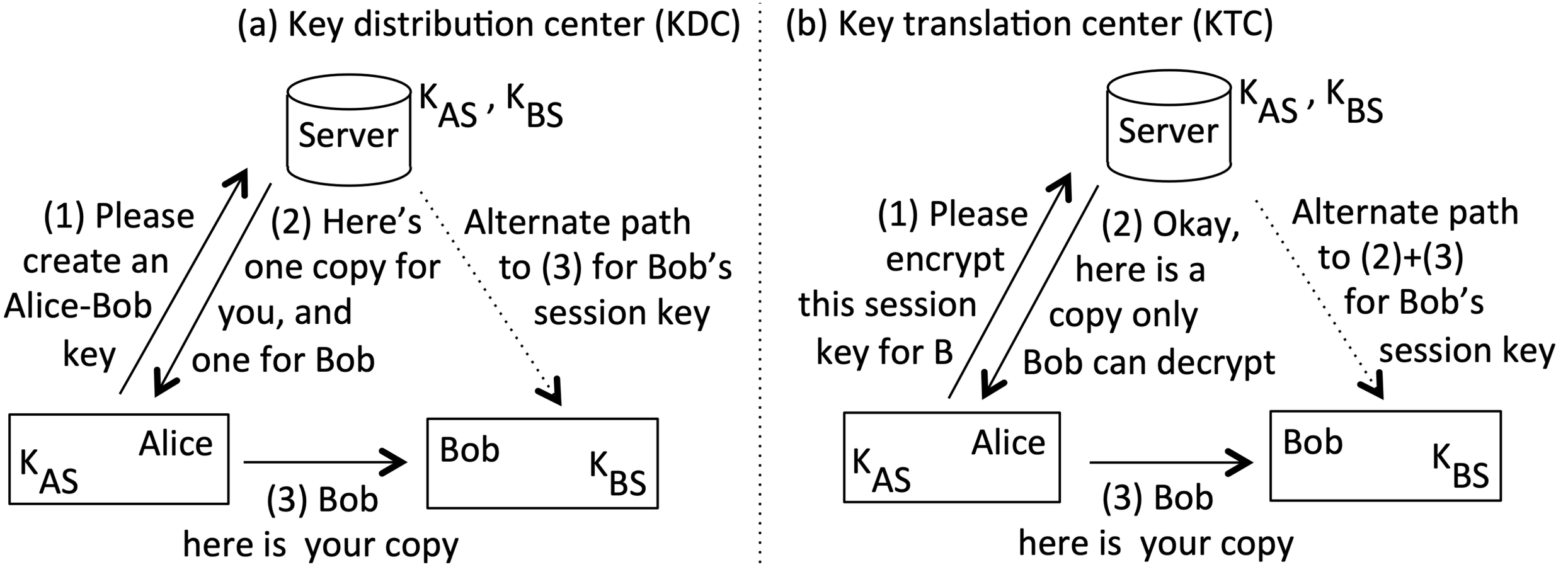
Key Management

Challenges

– Establishing and securing shared keys 🔐 .
– Managing keys for data at rest and in communications.

Reusing Session Keys

– Poor cryptographic hygiene to reuse permanent session keys.


(a) Key distribution center (KDC)
Server
K_{AS} , K_{BS}
(1) Please create an Alice-Bob key
(2) Here's one copy for you, and one for Bob
Alternate path to (3) for Bob's session key
K_{AS}
Alice
Bob
K_{BS}
(3) Bob here is your copy
(b) Key translation center (KTC)
Server
K_{AS} , K_{BS}
(1) Please encrypt this session key for B
(2) Okay, here is a copy only Bob can decrypt
Alternate path to (2)+(3) for Bob's session key
K_{AS}
Alice
Bob
K_{BS}
(3) Bob here is your copy

Authentication Protocols

Concepts and Mistakes

Demonstrating Knowledge of Secret

A basic idea used to authenticate a remote party B is to (a) associate a secret with B; and then (b) carry out a communication believed to be with B, accepting a demonstration of knowledge of that secret (key) as evidence that B is the party involved in the communication.

Replay Attack

– Attackers capture and replay a message without knowing the secret.

Example: Simple replay of `H(S)` (hash of the secret) in an authentication protocol.

Dictionary Attack

– Offline guessing attack using verifiable text.

Example: Using $H(r_A, W)$ for dictionary attack.

Reflection and Relay Attacks

Reflection Attack

– A reflection attack is a cryptographic attack where an adversary reuses a challenge issued by a verifier and reflects it back as a response, exploiting mutual authentication to impersonate a legitimate entity without solving the challenge.

Replay attack

Imagine someone recording your voice saying "I want pizza" and then playing that recording to the pizza place to order a pizza.

Reflection attack

An attacker approaches a security guard who asks for ID. Instead of providing one, the attacker reflects the request, making the guard show his own ID, which the attacker then reuses to gain access.

Relay Attack

– Relaying messages in real time to impersonate a party.

  Example: Car unlocking system using RF signals.

Common Attacks on Authentication Protocols

Types of Attacks

- Replay, reflection, relay, interleaving, dictionary, forward search, pre-capture.

Attack	Short description
replay	reusing a previously captured message in a later protocol run
reflection	replaying a captured message to the originating party
relay	forwarding a message in real time from a distinct protocol run
interleaving	weaving together messages from distinct concurrent protocols
middle-person	exploiting use of a proxy between two end-parties
dictionary	using a heuristically prioritized list in a guessing attack
forward search	feeding guesses into a one-way function, seeking output matches
pre-capture	extracting client OTPs by social engineering, for later use

Defense Mechanisms

- Use of time-variant parameters (TVPs).
  - Random Numbers
  - Sequence Number
  - Timestamps

Establishing Shared Keys by Public Agreement (DH)

Diffie-Hellman Key Exchange

What is Diffie-Hellman?

- A cryptographic protocol for securely exchanging a **shared secret** over an insecure channel.
- Used in encryption protocols like **TLS, SSH, and VPNs**.

Why is it Needed?

- If two parties (Alice & Bob) want to communicate securely, they need a **shared key**.
- Sending the key directly is insecure—an eavesdropper (Eve) could intercept it.
- **Diffie-Hellman allows them to derive the same key without ever sending it.**

Diffie-Hellman Key Exchange Protocol

- **Step 1: Public Parameters**
  - Alice and Bob agree on:
    - A large **prime number** (p)
    - A **generator** (g) (a primitive root modulo (p))
- **Step 2: Key Exchange**
  1. **Alice** picks a secret number (a) and sends $(g^a \mod p)$ to Bob.
     - **A** → **B**: $(g^a \mod p)$
  2. **Bob** picks a secret number (b) and sends ($g^b \mod p$) to Alice.
     - **A** ← **B**: ($g^b \mod p$)
- **Step 3: Compute the Shared Secret**
  - Bob calculates: $K = (g^a)^b \mod p = g^{ab} \mod p$
  - Alice calculates: $K = (g^b)^a \mod p = g^{ba} \mod p$
- Since **multiplication is commutative**, both Alice and Bob derive the same secret key.

Diffie-Hellman Key Agreement

- **Why is it Secure?**
  - An attacker (Eve) can see ($g^a \mod p$) and ($g^b \mod p$) but cannot compute ($g^{ab} \mod p$).
  - This is because of the **Discrete Logarithm Problem (DLP)**:
    - Given ($g^x \mod p$), it is computationally hard to determine (x).

Key Authentication Properties and Goals

Protocol Goals

- Arrange shared secret keys that are fresh, sufficiently long, and random.
- Ensure keys are known by both parties involved.

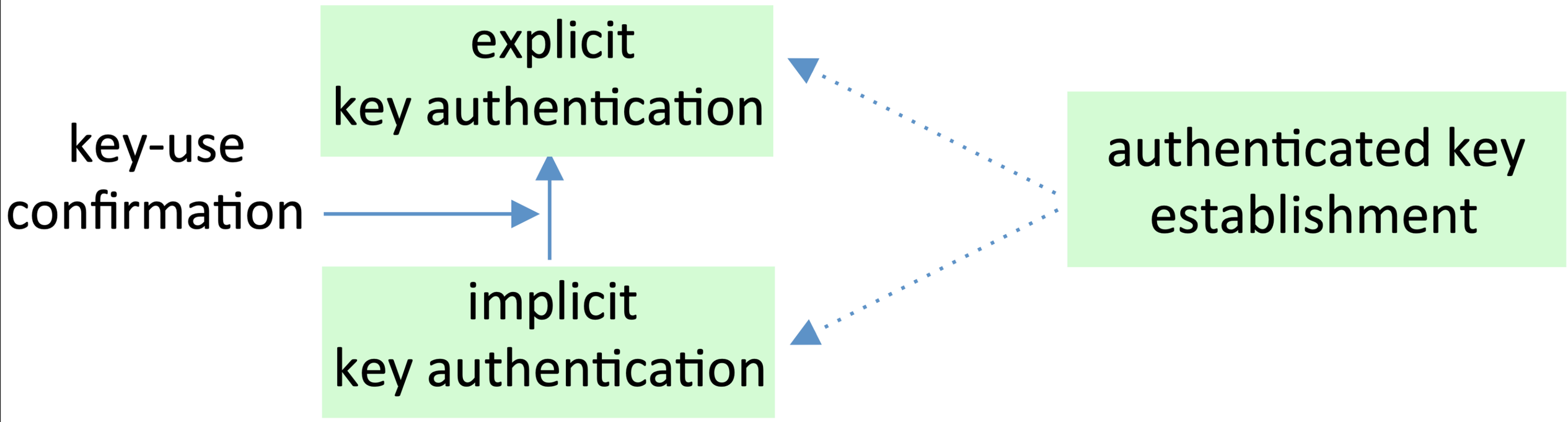
Forward Secrecy

- Disclosure of long-term keys does not compromise previous session keys.

Key Authentication Terminology

Implicit and Explicit Key Authentication

- **Implicit**: Key access scope is narrowed but not confirmed.
- **Explicit**: Key-use confirmation proves possession.


explicit
key authentication
key-use
confirmation
implicit
key authentication
authenticated key
establishment

Password-Authenticated Key Exchange

EKE and SPEKE

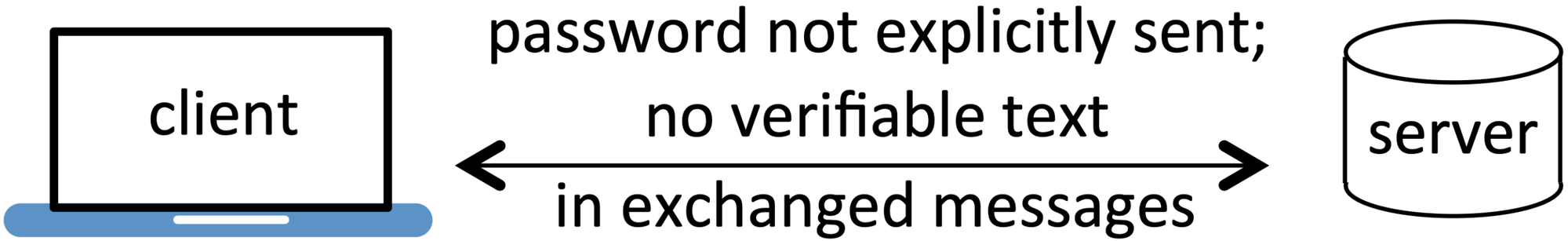
PAKE Protocols

PAKE (Password-authenticated key exchange)

Goals and Motivation

– Establish authenticated session keys using *weak passwords*.
– Resist offline password-guessing attacks.

Session key establishment with mutual authentication
based on weak password *w* (only parties knowing w can compute *K*)



start with shared *w*;
end with fresh session key *K*

start with shared *w*;
end with fresh session key *K*

Encrypted Key Exchange (EKE)

Basic EKE Protocol

1. A → B: A, $\{ e_A \}$w
2. A ← B: $\{ E_{eA}(K) \}$w
3. A → B: {T}k

Analysis

– Attackers cannot verify guesses for W without knowing K.

Diffie-Hellman EKE (DH-EKE)

Protocol Steps

1. A $\rightarrow$ B: A, $\{ g^a \}$W
2. A $\leftarrow$ B: $\{ g^b \}$W
3. A and B compute K from DH agreement.

Forward Secrecy

– Provides forward secrecy using fresh keys for each session.

Single Sign-On (SSO) and Federated Identity Systems

Single Sign-On (SSO)

- Allows users to authenticate once and gain access to multiple systems.
- Reduces password fatigue and increases usability.

Types of SSO Systems:

1. Credential manager (CM)
2. Enterprise SSO
3. Federated identity

Federated Identity Systems

- Enable identity federation across different domains and organizations.
- **Examples**: SAML, OAuth, OpenID Connect.